

HYPERHESSIANS.JL

# Forward-mode AD specialized for second-order derivatives

Kristoffer Carlsson

@KristofferC

✉ kristoffer.carlsson@juliahub.com

JuliaCon 2026, Mainz

slides: [kristoffer.github.io/HyperHessians.jl/slides](https://kristoffer.github.io/HyperHessians.jl/slides)



# Theory

# Automatic Differentiation

- What is Automatic Differentiation (AD)?
- Obtain numerically "exact" derivatives of "subroutines" without having to implement them manually.
- Saves time and avoids bugs.
- Not symbolic differentiation, not expression building
- Reverse vs forward mode differentiation
- This talk: **forward mode**

# Finite differences

Taylor, with a step  $h$

$$f(x_0 + h) = f(x_0) + h f'(x_0) + \frac{h^2}{2} f''(x_0) + \dots$$

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} + \mathcal{O}(h)$$

- Big  $h$ : the dropped Taylor terms dominate.
- Small  $h$ : the subtraction cancels in floating point.
- Have to call `f` twice

`f(x) = x sin(x2)` at `x = 2`, error vs exact `f'`

```
julia> f(x) = x * sin(x * x);
```

```
julia> fd(f, x, h) = (f(x + h) - f(x)) / h;
```

```
julia> exact = sin(4.0) + 8cos(4.0);
```

```
julia> fd(f, 2.0, 1e-2) - exact  
0.08441453688334022      # truncation
```

```
julia> fd(f, 2.0, 1e-14) - exact  
-0.14247963371404015    # cancellation
```



# Definition and operators

- Complex numbers:  $z = a + bi$ ,  $i^2 = -1$ ,  $\text{Im}[z] = b$
- Dual numbers:  $d = x + h\varepsilon$ ,  $\varepsilon^2 = 0$ ,  $\varepsilon[d] = h$

$$f(d) = f(x + h\varepsilon) = f(x) + hf'(x)\varepsilon + \underbrace{\frac{h^2}{2}f''(x)\varepsilon^2 + \dots}_{= 0 \text{ exactly}}$$

$$f'(x) = \varepsilon[f(x + h\varepsilon)]/h$$

- Primitives need one rule each:  $\sin(d) = \sin(x) + h \cos(x) \varepsilon$ .
- $d_1 d_2 = x_1 x_2 + (x_1 h_2 + x_2 h_1) \varepsilon$ : the product rule falls out ( $h_1 h_2 \varepsilon^2 = 0$ ).
- Seed  $h = 1$ , read the derivative off the  $\varepsilon$  slot:  $f'(x) = \varepsilon[f(x + \varepsilon)]$ .

# Implementation

## Forward mode AD in Julia

```

struct Dual
    x::Float64  # value
    ε::Float64  # epsilon coefficient
end
Base.:+(a::Dual, b::Dual) = Dual(a.x + b.x, a.ε + b.ε)
Base.:*(a::Dual, b::Dual) = Dual(a.x * b.x, a.x * b.ε + b.x * a.ε) # product rule
Base.sin(d::Dual)         = Dual(sin(d.x), cos(d.x) * d.ε)         # chain rule
derivative(f, x) = f(Dual(x, 1.0)).ε # seed ε = 1, run f, read ε

```

```
julia> f(x) = x * sin(x * x);
```

```
julia> derivative(f, 2.0)
-5.985951462216824
```

```
julia> sin(4) + 8cos(4)
-5.985951462216824
```

```

julia> @code_warntype optimize=true derivative(f, 2.0)
...
  x::Float64
Body::Float64
1 ~ %1 = intrinsic Base.mul_float(x, x)::Float64
|   %2 = intrinsic Base.mul_float(x, 1.0)::Float64
|   %3 = intrinsic Base.mul_float(x, 1.0)::Float64
|   %4 = intrinsic Base.add_float(%2, %3)::Float64
|   %5 =      invoke Main.sin(%1::Float64)::Float64
|   %6 =      invoke Main.cos(%1::Float64)::Float64
...

```

- Data layout: ■■ where ■ value ■  $\epsilon$  coefficient

# Higher order derivatives

- derivative is just Julia code, we know how to differentiate that!
- second\_derivative(f, x) = derivative(y -> derivative(f, y), x)
- Nested Dual numbers

```
struct Dual
    x::Float64
    ε::Float64
end
derivative(f, x) = f(Dual(x, 1.0)).ε
```

```
struct Dual{T}
    x::T
    ε::T
end
derivative(f, x) = f(Dual(x, one(x))).ε
Base.one(d::Dual) = Dual(one(d.x), zero(d.x))
Base.cos(d::Dual) = Dual(cos(d.x), -sin(d.x) * d.ε)
```

```
julia> second_derivative(f, x) = derivative(y -> derivative(f, y), x);
```

```
julia> second_derivative(f, 2.0)
16.37395639949036
```

```
julia> 12cos(4) - 32sin(4)
16.37395639949036
```

- Data layout:   $(f, f')$    $(f', f'')$

# Jacobians: $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$

$$f(\mathbf{x} + \mathbf{h} \varepsilon) = f(\mathbf{x}) + J(\mathbf{x}) \mathbf{h} \varepsilon, \quad \mathbf{x}, \mathbf{h} \in \mathbb{R}^n, \quad f(\mathbf{x}) \in \mathbb{R}^m, \quad J(\mathbf{x}) \in \mathbb{R}^{m \times n}, \quad J_{kj} = \frac{\partial f_k}{\partial x_j}$$

- One pass is a **Jacobian-vector product**. Seed column by column:  $\mathbf{h} = \mathbf{e}_i$  picks out  $J\mathbf{e}_i$ : column  $i$ .

n passes, one per column

```
function jacobian(f, x)
    cols = []
    for i in eachindex(x)
        seed = zeros(length(x))
        seed[i] = 1.0 # direction e_i
        y = f(Dual.(x, seed))
        push!(cols, [d.ε for d in y])
    end
    return stack(cols) # m × n
end
```

finite differences: same loop, n + 1 evaluations

```
function jacobian_fd(f, x; h = 1e-8)
    f0 = f(x)
    cols = []
    for i in eachindex(x)
        xh = copy(x)
        xh[i] += h
        push!(cols, (f(xh) - f0) / h)
    end
    return stack(cols) # m × n
end
```



# Chunk mode: N columns per pass

$$f(\mathbf{x} + H \varepsilon) = f(\mathbf{x}) + J(\mathbf{x}) H \varepsilon, \quad H \in \mathbb{R}^{n \times N} \text{ — one direction per column, } H = I \Rightarrow JH = J$$

scalar partial → chunk of partials

|                                                                                                                                                                              |                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> struct Dual{T}   x::T   ε::T end Base.:(a::Dual, b::Dual) =   Dual(a.x * b.x,         a.x * b.ε + b.x * a.ε) Base.sin(d::Dual) = Dual(sin(d.x), cos(d.x) * d.ε) </pre> | <pre> struct Dual{T,N}   x::T   ε::NTuple{N,T} end Base.:(a::Dual, b::Dual) =   Dual(a.x * b.x,         a.x .* b.ε .+ b.x .* a.ε) Base.sin(d::Dual) = Dual(sin(d.x), cos(d.x) .* d.ε) </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- Seed the identity all at once ( $H = I$ , so  $N = n$ ) → whole Jacobian in **one pass**.
- `sin` still calls `cos` once: expensive rule work is amortized across all  $N$  directions.
- Cost per op grows with  $N$ , a fat `Dual{N}` spills registers → cap  $N$  and sweep  $n/N$  passes (more on this later).
- Data layout: `value ε-tuple`,  $N = 4$  partials

# Chunk mode: the code

$H = I$ : whole Jacobian, one call to `f`

```
function jacobian(f, x)
    n = length(x)
    ds = []
    for i in 1:n
        seed = zeros(n)
        seed[i] = 1.0      # row i of H = I
        push!(ds, Dual(x[i], Tuple(seed)))
    end
    y = f(ds)              # ONE call
    return stack([d.ε for d in y], dims = 1)  # m × n
end
```

primal computed once

large input -> fat duals

```
julia> F(x) = [x[1] * sin(x[2]), x[1] * x[2]];
```

```
julia> jacobian(F, [1.0, 2.0])
2×2 Matrix{Float64}:
 0.909297 -0.416147
 2.0      1.0
```

chunk mode:  $n/N$  passes

```
function jacobian(f, x, N)
    n = length(x)
    blocks = []
    for c in 0:N:n-1      # one pass per chunk
        ds = []
        for i in 1:n
            seed = zeros(N)
            if c < i <= c + N # chunk c of row i
                seed[i-c] = 1.0
            end
            push!(ds, Dual(x[i], Tuple(seed)))
        end
        y = f(ds)          # J[:, c+1:c+N]
        push!(blocks, stack([d.ε for d in y], dims = 1))
    end
    return hcat(blocks...) # m × n
end
```

lean duals

primal recomputed  $n/N$  times

# Hessians

- The Hessian is the Jacobian of the gradient
- We can do Jacobians, we can do gradients...

```
gradient(f, x) = vec(jacobian(y -> [f(y)], x))

hessian(f, x) = jacobian(y -> gradient(f, y), x)
```

```
julia> f(x) = x[1] * sin(x[2] * x[3]);

julia> hessian(f, [1.0, 2.0, 3.0])
3×3 Matrix{Float64}:
 0.0      2.88051  1.92034
 2.88051  2.51474  2.63666
 1.92034  2.63666  1.11766
```

- The seeds nest exactly like the scalar case.
- Memory layout,  $N = 3$ : , in field order:  $(f, \nabla_1)$ , then each  $\varepsilon$  slot  $(\nabla_2^{(j)}, H_{j,:})$

# Definition and operators

- Dual numbers, one direction:  $d = x + h\varepsilon$ ,  $\varepsilon^2 = 0$  — first order only.
- HyperDual numbers:  $d = x + a\varepsilon_1 + b\varepsilon_2 + c\varepsilon_1\varepsilon_2$ ,  $\varepsilon_1^2 = \varepsilon_2^2 = 0$ ,  $\varepsilon_1\varepsilon_2 \neq 0$

$$f(d) = f(x) + f'(x)\delta + \frac{f''(x)}{2}\delta^2 + \dots, \quad \delta = a\varepsilon_1 + b\varepsilon_2 + c\varepsilon_1\varepsilon_2$$

$$\delta^2 = 2ab\varepsilon_1\varepsilon_2, \quad \delta^3 = 0 \text{ exactly} \Rightarrow$$

$$f(d) = f(x) + af'(x)\varepsilon_1 + bf'(x)\varepsilon_2 + (cf'(x) + abf''(x))\varepsilon_1\varepsilon_2$$

- Second-order chain rule pops out.
- Again, one rule per primitive

$$\sin(d) = \sin(x) + a\cos(x)\varepsilon_1 + b\cos(x)\varepsilon_2 + (c\cos(x) - ab\sin(x))\varepsilon_1\varepsilon_2$$

- $d_1d_2 = x_1x_2 + (x_1a_2 + x_2a_1)\varepsilon_1 + (x_1b_2 + x_2b_1)\varepsilon_2 + (x_1c_2 + x_2c_1 + a_1b_2 + a_2b_1)\varepsilon_1\varepsilon_2$  — the product rule falls out again.
- Seed  $a = b = 1, c = 0$ , read the  $\varepsilon_1\varepsilon_2$  slot:  $f''(x) = \varepsilon_1\varepsilon_2[f(x + \varepsilon_1 + \varepsilon_2)]$ .

# Implementation

```

struct HyperDual
    x::Float64
    ε1::Float64
    ε2::Float64
    ε12::Float64
end

Base.*(a::HyperDual, b::HyperDual) =
    HyperDual(a.x * b.x,
              a.x * b.ε1 + b.x * a.ε1,
              a.x * b.ε2 + b.x * a.ε2,
              a.x * b.ε12 + b.x * a.ε12 + a.ε1 * b.ε2 + a.ε2 * b.ε1) # product rule

function Base.sin(d::HyperDual)
    s, c = sincos(d.x)
    HyperDual(s, c * d.ε1, c * d.ε2, c * d.ε12 - s * d.ε1 * d.ε2)
end

second_derivative(f, x) = f(HyperDual(x, 1.0, 1.0, 0.0)).ε12 # a = b = 1, c = 0

```

```

julia> f(x) = x * sin(x * x);

julia> second_derivative(f, 2.0)
16.37395639949036

julia> 12cos(4) - 32sin(4)
16.37395639949036

```

- Data layout:  where  value   $f'$    $f'$  again   $f''$

# Hessian: one entry per pass

The seeds pick *which* entry:  $\varepsilon_1 = \mathbf{e}_i, \varepsilon_2 = \mathbf{e}_j$  reads  $H[i, j]$  off the  $\varepsilon_1 \varepsilon_2$  slot

**$n(n+1)/2$  passes, one entry each**

```
function hessian(f, x)
    n = length(x)
    H = zeros(n, n)
    for i in 1:n, j in i:n # symmetry: i ≤ j only
        ds = [HyperDual(x[k], float(k == i),
                        float(k == j), 0.0)
              for k in 1:n]
        H[i, j] = H[j, i] = f(ds).ε12 # one pass
    end
    return H
end
```

```
julia> f(x) = x[1] * sin(x[2] * x[3]);
```

```
julia> hessian(f, [1.0, 2.0, 3.0])
```

```
3×3 Matrix{Float64}:
```

```
0.0      2.88051  1.92034
2.88051  2.51474  2.63666
1.92034  2.63666  1.11766
```

- The scalar `HyperDual` from the last slide
- $H$  is symmetric, so only  $i \leq j$ :  $\frac{n(n+1)}{2}$  evaluations instead of  $n^2$ .
- The primal ( `f` ) and every rule are recomputed for **every entry**.

# Chunk mode: an $N_1 \times N_2$ block of $H$ per pass

An off-diagonal Hessian block mixes directions from two different chunks

$$d = x + \sum_i a_i \varepsilon_{1,i} + \sum_j b_j \varepsilon_{2,j}, \quad [\varepsilon_{1,i} \ \varepsilon_{2,j}] f(d) = \frac{\partial^2 f}{\partial x_i \partial x_j} = H[i, j]$$

```
struct HyperDual
```

```
  x::Float64
```

```
  ε1::Float64
```

```
  ε2::Float64
```

```
  ε12::Float64
```

```
end
```

```
HyperDual(s, c * d.ε1, c * d.ε2,  
          c * d.ε12 - s * d.ε1 * d.ε2)
```

```
struct HyperDual{N1,N2}
```

```
  x::Float64
```

```
  ε1::NTuple{N1,Float64}
```

```
  ε2::NTuple{N2,Float64}
```

```
  ε12::NTuple{N1,NTuple{N2,Float64}}
```

```
end
```

```
HyperDual(s, c .* d.ε1, c .* d.ε2, tuple(i ->  
      c .* d.ε12[i] .- s .* (d.ε1[i] .* d.ε2),  
      N1))
```

- $\varepsilon_1$  carries chunk  $I$ ,  $\varepsilon_2$  carries chunk  $J$ , the  $\varepsilon_1 \varepsilon_2$  slots are an  $N_1 \times N_2$  **block** of  $H$ .
- `HyperDual{N1,N2}` :  $1 + N_1 + N_2 + N_1 N_2$  slots — `{8,8}` is 81 floats

# Chunk mode: the code

$N_1 = N_2 = n$ : whole Hessian, one call to  $f$

```
seed(i, R) = Tuple(float(i == j) for j in R)

function hessian(f, x)                #  $N_1 = N_2 = n$ 
    n = length(x)
    ds = [HyperDual(x[i], seed(i, 1:n), seed(i, 1:n))
           for i in 1:n]              #  $\epsilon_{12}$  seeded to zero
    v = f(ds)                         # ONE call
    return [v.ε12[i][j] for i in 1:n, j in 1:n]
end
```

primal & rules once

$n^2$  slots per number

```
julia> f(x) = x[1] * sin(x[2] * x[3]);
```

```
julia> hessian(f, [1.0, 2.0, 3.0])
```

3×3 Matrix{Float64}:

```
0.0      2.88051  1.92034
2.88051  2.51474  2.63666
1.92034  2.63666  1.11766
```

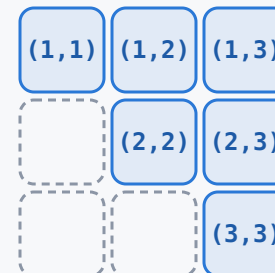
Chunked mode: one block pair per pass

```
function hessian(f, x, N)
    n = length(x); H = zeros(n, n)
    for s1 in 1:N:n, s2 in s1:N:n      # block pairs,  $I \leq J$ 
        I, J = s1:s1+N-1, s2:s2+N-1
        ds = [HyperDual(x[i], seed(i, I), seed(i, J))
               for i in 1:n]
        v = f(ds)                      # one pass per pair
        H[I, J] .= [v.ε12[i][j] for i in 1:N, j in 1:N]
    end
    return symmetrize!(H)              # mirror  $I < J$  blocks
end
```

lean numbers

symmetry:  $k(k+1)/2$  passes

primal recomputed per pair



$H$  block by block,  $k = 3$  chunks: one evaluation pass per chunk pair  $(I, J)$ ,  $I \leq J$  — the dashed lower blocks come free from `symmetrize!`



# Same four groups of data

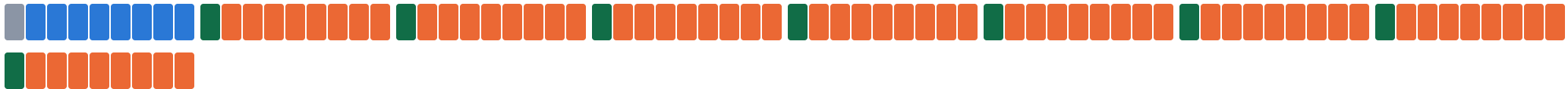
Nested duals and hyperdual numbers hold the same 81 floats in a different layout

■ value   ■ gradient chunk 1 (8)   ■ gradient chunk 2 (8)   ■ Hessian block (8×8)

**HyperDual{8,8}** — each group contiguous, blocks: [1, 8, 8, 8×8]



**nested Dual 8×8** — one gradient float interleaved into every Hessian row, blocks: [1, 8, [1, 8]×8]



# Implementations

# ForwardDiff.jl

The "GOAT" (Greatest of All Time): the standard forward mode package since 2015; its `Dual{Tag,V,N}` is the chunked  $\varepsilon$ -tuple we just built

```
julia> using ForwardDiff

julia> f(x) = x * sin(x * x);

julia> ForwardDiff.derivative(f, 2.0)
-5.985951462216824

julia> F(x) = [x[1] * sin(x[2]), x[1] * x[2]];

julia> ForwardDiff.jacobian(F, [1.0, 2.0])
2×2 Matrix{Float64}:
 0.909297 -0.416147
 2.0      1.0
```

```
julia> g(x) = x[1] * sin(x[2] * x[3]);

julia> x = [1.0, 2.0, 3.0];

julia> ForwardDiff.gradient(g, x)
3-element Vector{Float64}:
-0.27941549819892586
 2.880510859951098
 1.920340573300732

julia> ForwardDiff.hessian(g, x)
3×3 Matrix{Float64}:
 0.0      2.88051  1.92034
 2.88051  2.51474  2.63666
 1.92034  2.63666  1.11766
```

- The `Tag` type parameter guards nested calls against perturbation confusion
- Preallocation + chunk picking: `hessian(g, x, HessianConfig(g, x, Chunk{4}()))`.
- `hessian` implementation is the nested `Dual` jacobian-of-gradient
- No JVP (Jacobian vector product) or HVP API

# JVP + HVP with DifferentiationInterface.jl

- ForwardDiff has no native JVP or HVP entry point
- DifferentiationInterface provides them on top of it: `pushforward` (the JVP) and `hvp`

```
julia> using DifferentiationInterface; import ForwardDiff

julia> backend = AutoForwardDiff(); # or AutoEnzyme(), AutoZygote(), AutoFiniteDiff(), ...

julia> pushforward(F, backend, [1.0, 2.0], ([0.1, 0.2],)) # JVP: J(x) v, one dual pass
([0.007700375373139695, 0.4],)

julia> v = [0.1, 0.2, 0.3]; hvp(g, backend, x, (v,)) # HVP: H(x) v, nested duals
([1.1522043439804392, 1.5819979655063527, 1.0546653103375685],)
```

# HyperHessians.jl

- Experiment: How would an AD package based on hyperdual numbers compare with ForwardDiffs nested duals?
- Initial commit `@KristofferC` on Nov 29, 2021 (vibeless!)
- Got some results showing it was faster, but no time to make something actually useful out of it.
- Recent improvements in agentic coding changed the cost/benefit analysis. Now actually reasonable to create a useful package

## API

```

julia> using HyperHessians: hessian, hessian!, hvp, hvhp

julia> g(x) = x[1] * sin(x[2] * x[3]);

julia> x = [1.0, 2.0, 3.0]; v = [0.1, 0.2, 0.3];

julia> hessian(g, x)
3×3 Matrix{Float64}:
 0.0      2.88051  1.92034
 2.88051  2.51474  2.63666
 1.92034  2.63666  1.11766

julia> hvp(g, x, v)
3-element Vector{Float64}:
 1.1522043439804392
 1.5819979655063527
 1.0546653103375685

julia> hvhp(g, x, v)          # v' H(x) v
0.748019620600585

```

```

julia> using HyperHessians: HessianConfig, Chunk, Jet

julia> cfg = HessianConfig(x, Chunk{3}(); simd = true);

julia> H = zeros(3, 3);

julia> hessian!(H, g, x, cfg);      # reuse cfg and H

julia> using HyperHessians: hessian_gradient_value

julia> r = hessian_gradient_value(g, x, cfg);

julia> r.value
-0.27941549819892586

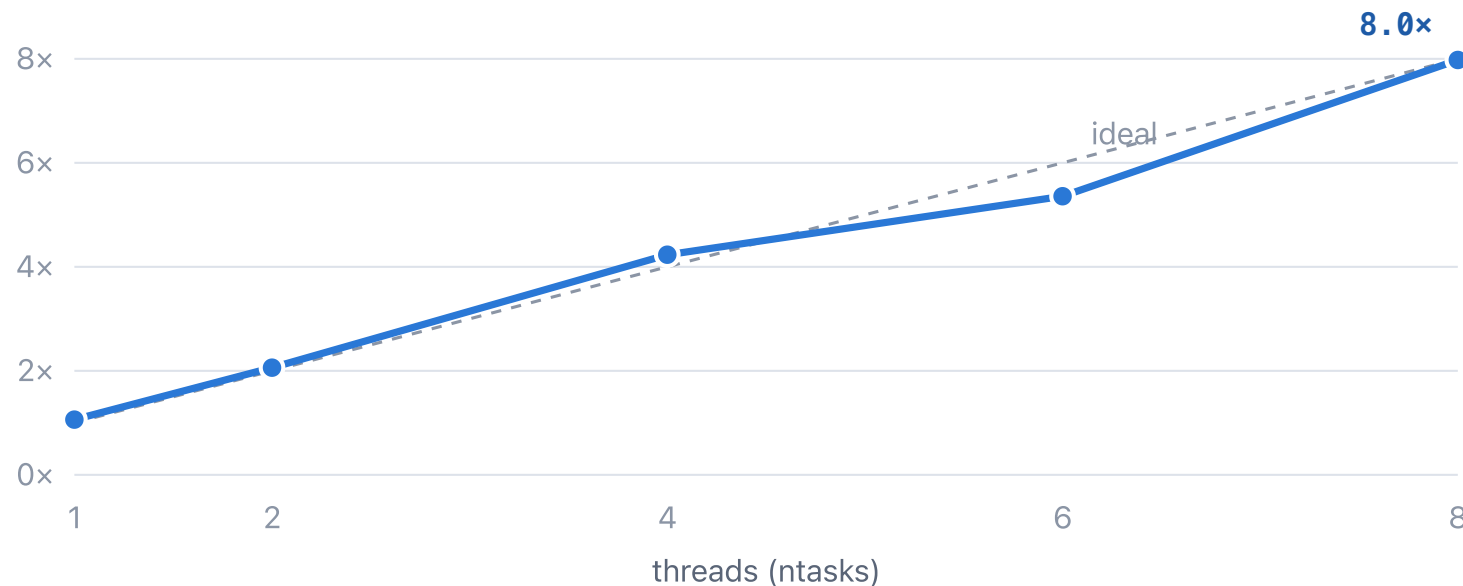
julia> r.gradient
3-element Vector{Float64}:
 -0.27941549819892586
  2.880510859951098
  1.920340573300732

```

- `*_gradient_value`: value and gradient sit in slots the pass already computed — **free**.

# Threading

```
cfg = HyperHessians.ThreadedHessianConfig(x, Chunk{8}()) # ntasks = Threads.nthreads()
HyperHessians.hessian!(H, g, x, cfg)
```



ackley,  $n = 512$ , `Chunk{8}`, `julia -t N` on Apple M4 Pro, BenchmarkTools minimum — speedup vs the serial `HessianConfig`

- `f` must be safe to call concurrently: no shared closed-over buffers, side effects in any order.

# HyperHessians through DifferentiationInterface

AutoHyperHessians is a registered ADTypes backend

```
julia> using DifferentiationInterface

julia> import HyperHessians      # activates the AutoHyperHessians extension

julia> backend = AutoHyperHessians();    # or (chunksize = 4, simd = true), or (jet = true)

julia> hessian(g, backend, x)
3×3 Matrix{Float64}:
 0.0      2.88051  1.92034
 2.88051  2.51474  2.63666
 1.92034  2.63666  1.11766

julia> hvp(g, backend, x, (v,))
([1.1522043439804392, 1.5819979655063527, 1.0546653103375685],)
```



# Benchmarking

# ChunkPicker.jl

- Best chunk size depends on hardware (register sizes, AVX512, etc), input function, input length, etc
- Need to measure

```
julia> res = pick_chunk(HyperHessiansBackend(), ackley, x;    # x = rand(32)
                        op = :hessian)
ChunkPickResult (HyperHessians, :hessian)
...
  chunk 4 simd      17.208 μs  1.25x
  chunk 6           26.625 μs  1.94x
  chunk 6 simd      21.875 μs  1.59x
  chunk 8           23.084 μs  1.68x
* chunk 8 simd      13.750 μs  1.00x
  chunk 11          27.000 μs  1.96x
  chunk 11 simd     17.875 μs  1.30x
  chunk 12          31.292 μs  2.28x
...
→ HyperHessians.HessianConfig(x, HyperHessians.Chunk{8}()); simd = true)
```

- Also works for ForwardDiff and other operators: `:gradient`, `:jacobian`, `:hessian`, `:hvp`.

# Hessians — Apple M4 Pro

Speedup = ForwardDiff time / HyperHessians time, each at its picked best configuration

| FUNCTION            | N=4             | N=16             | N=64            | N=256            | GEOMEAN |
|---------------------|-----------------|------------------|-----------------|------------------|---------|
| ackley              | 2.52×<br>Js/c4  | 2.30×<br>c4s/c16 | 3.26×<br>c8s/c8 | 3.63×<br>c8s/c16 | 2.88×   |
| rosenbrock          | 2.86×<br>c4s/c4 | 3.43×<br>c4s/c8  | 3.94×<br>c4s/c8 | 4.45×<br>c6s/c6  | 3.62×   |
| logsumexp           | 2.64×<br>Js/c4  | 2.37×<br>J/c8    | 2.34×<br>c6/c8  | 2.79×<br>c6/c8   | 2.53×   |
| self_weighted_logit | 1.74×<br>J/c4   | 2.16×<br>J/c16   | 2.36×<br>c8s/c4 | 2.38×<br>c8s/c16 | 2.14×   |
| geomean             | 2.40×           | 2.52×            | 2.90×           | 3.22×            | 2.74×   |

per cell: HyperHessians pick / ForwardDiff pick — cN = chunk N, s = simd, J = Jet

# Hessians — AMD EPYC 9354, AVX-512

| FUNCTION            | N=4             | N=16             | N=64              | N=256             | GEOMEAN |
|---------------------|-----------------|------------------|-------------------|-------------------|---------|
| ackley              | 2.47×<br>Js/c4  | 2.31×<br>J/c16   | 2.86×<br>c16s/c11 | 3.61×<br>c16s/c16 | 2.77×   |
| rosenbrock          | 4.27×<br>c4s/c4 | 2.90×<br>c4s/c16 | 4.11×<br>c8s/c16  | 4.85×<br>c8s/c16  | 3.96×   |
| logsumexp           | 1.98×<br>c4/c4  | 1.82×<br>J/c16   | 1.90×<br>c4/c16   | 2.02×<br>c16s/c16 | 1.93×   |
| self_weighted_logit | 1.59×<br>c4s/c4 | 1.94×<br>J/c16   | 1.77×<br>c16s/c13 | 2.58×<br>c16s/c16 | 1.94×   |
| geomean             | 2.40×           | 2.20×            | 2.51×             | 3.09×             | 2.53×   |

per cell: HyperHessians pick / ForwardDiff pick — cN = chunk N, s = simd, J = Jet

# HVPs — Apple M4 Pro

| FUNCTION            | N=4             | N=16              | N=64             | N=256            | GEOMEAN |
|---------------------|-----------------|-------------------|------------------|------------------|---------|
| ackley              | 1.52×<br>c4s/c4 | 1.97×<br>c16s/c16 | 1.78×<br>c16s/c8 | 1.94×<br>c16s/c8 | 1.80×   |
| rosenbrock          | 2.31×<br>c4s/c4 | 1.73×<br>c16s/c2  | 1.86×<br>c8s/c2  | 2.10×<br>c16s/c2 | 1.99×   |
| logsumexp           | 1.59×<br>c4s/c4 | 1.56×<br>c16s/c16 | 1.39×<br>c16s/c8 | 1.49×<br>c16s/c8 | 1.50×   |
| self_weighted_logit | 1.44×<br>c4/c4  | 1.42×<br>c16/c4   | 1.21×<br>c8/c4   | 1.29×<br>c12/c4  | 1.34×   |
| geomean             | 1.69×           | 1.66×             | 1.54×            | 1.67×            | 1.64×   |

per cell: HyperHessians pick / ForwardDiff pick — cN = chunk N, s = simd, J = Jet

# HVPs — AMD EPYC 9354, AVX-512

| FUNCTION            | N=4             | N=16              | N=64              | N=256             | GEOMEAN |
|---------------------|-----------------|-------------------|-------------------|-------------------|---------|
| ackley              | 1.93×<br>c4s/c4 | 2.35×<br>c16s/c16 | 2.00×<br>c16s/c13 | 2.07×<br>c16s/c16 | 2.08×   |
| rosenbrock          | 2.14×<br>c4s/c4 | 1.93×<br>c4s/c16  | 1.93×<br>c4s/c13  | 1.84×<br>c4s/c16  | 1.96×   |
| logsumexp           | 1.63×<br>c4s/c4 | 2.62×<br>c16s/c16 | 2.61×<br>c16s/c22 | 2.87×<br>c16s/c16 | 2.38×   |
| self_weighted_logit | 1.19×<br>c4/c4  | 1.36×<br>c16/c16  | 1.10×<br>c16s/c11 | 1.18×<br>c16/c16  | 1.20×   |
| geomean             | 1.68×           | 2.00×             | 1.83×             | 1.90×             | 1.85×   |

per cell: HyperHessians pick / ForwardDiff pick — cN = chunk N, s = simd, J = Jet

# A real finite element modeling (FEM) problem

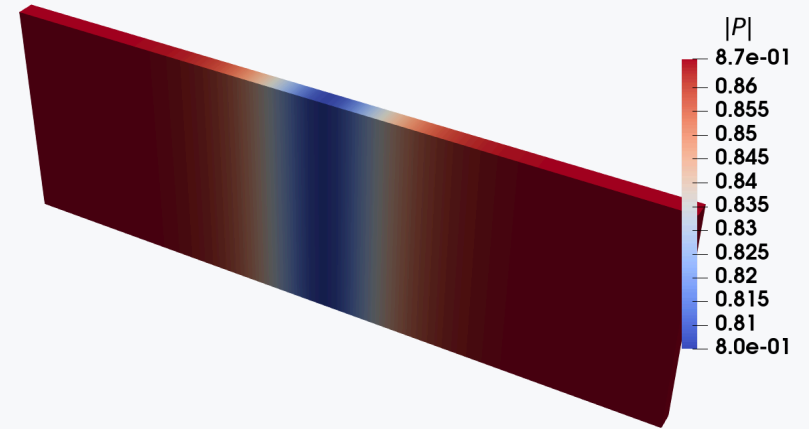
- [ferrite-fem.github.io/Ferrite.jl/stable/gallery/landau](https://ferrite-fem.github.io/Ferrite.jl/stable/gallery/landau)

```
# 4th order Landau free energy
function Fl(P::Vec{3, T}, α::Vec{3}) where {T}
    P2 = Vec{3, T}((P[1]^2, P[2]^2, P[3]^2))
    return α[1] * sum(P2) +
        α[2] * (P[1]^4 + P[2]^4 + P[3]^4) +
        α[3] * ((P2[1] * P2[2] + P2[2] * P2[3]) + P2[1] * P2[3])
end

# Ginzburg free energy
@inline Fg(∇P, G) = 0.5(∇P ⊠ G) ⊠ ∇P

# Ginzburg-Landau free energy
F(P, ∇P, params) = Fl(P, params.α) + Fg(∇P, params.G)

function element_potential(eldofs::AbstractVector{T},
                           cvP, params) where {T}
    energy = zero(T)
    for qp in 1:getnquadpoints(cvP)
        P = function_value(cvP, qp, eldofs)
        ∇P = function_gradient(cvP, qp, eldofs)
        energy += F(P, ∇P, params) * getdetJdV(cvP, qp)
    end
    return energy
end
```



# Landau

ChunkPicker on the element potential — actual example hardcoded `chunksize = 4`.

```
julia> pick_chunk(AutoForwardDiff(), potfunc, eldofs;
                op = :hessian)
ChunkPickResult (AutoForwardDiff, :hessian)
* chunk 2      41.250 μs  1.00x
  chunk 3      55.042 μs  1.33x
  chunk 4      46.292 μs  1.12x
  chunk 6      43.000 μs  1.04x
  chunk 8      74.958 μs  1.82x
  chunk 12     64.666 μs  1.57x
→ AutoForwardDiff(chunksize = 2)
```

```
julia> pick_chunk(AutoHyperHessians(), potfunc, eldofs;
                op = :hessian)
ChunkPickResult (AutoHyperHessians, :hessian)
* chunk 2      21.083 μs  1.00x
  chunk 3      32.416 μs  1.54x
  chunk 4      29.708 μs  1.41x
  chunk 6      31.791 μs  1.51x
  chunk 8      55.042 μs  2.61x
  chunk 12     62.333 μs  2.96x
→ AutoHyperHessians(chunksize = 2)
```

| GLOBAL HESSIAN ASSEMBLY          | TIME     | SPEEDUP |
|----------------------------------|----------|---------|
| AutoForwardDiff(chunksize = 2)   | 189.7 ms |         |
| AutoHyperHessians(chunksize = 2) | 98.0 ms  | 1.94x   |

23 409 dofs, 30 000 linear tetrahedra (12 dofs per element), 8 threads on Apple M4 Pro, BenchmarkTools minimum, assembled Hessians verified equal



# Same four groups of data

Nested duals and hyperdual number hold the same 81 floats in a different layout

■ value   ■ gradient chunk 1 (8)   ■ gradient chunk 2 (8)   ■ Hessian block (8x8)

HyperDual{8,8} — each group contiguous, blocks: [1, 8, 8, 8x8]



nested Dual 8x8 — one gradient float interleaved into every Hessian row, blocks: [1, 8, [1, 8]x8]

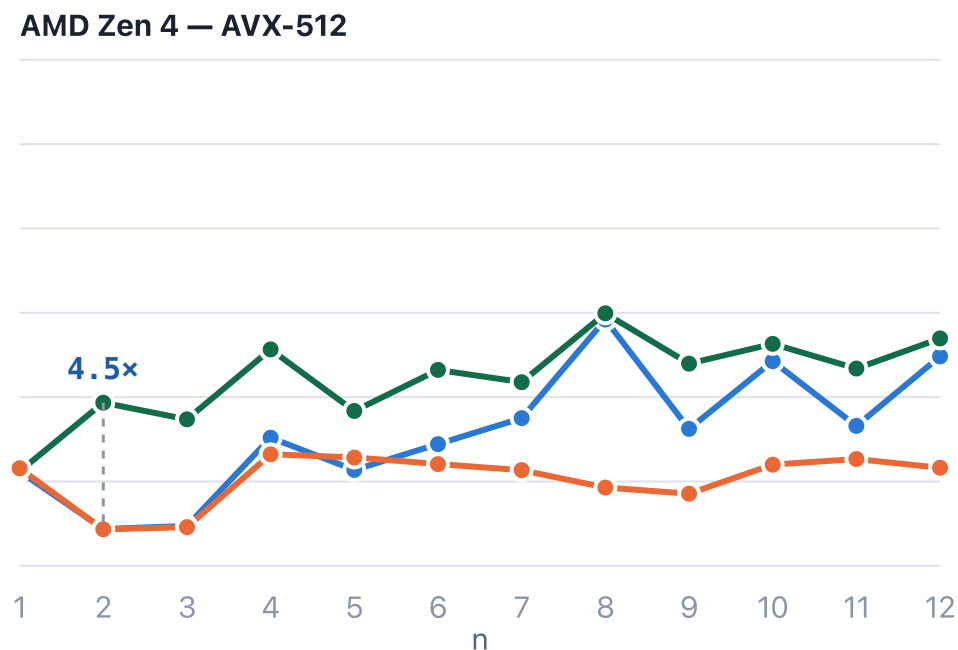
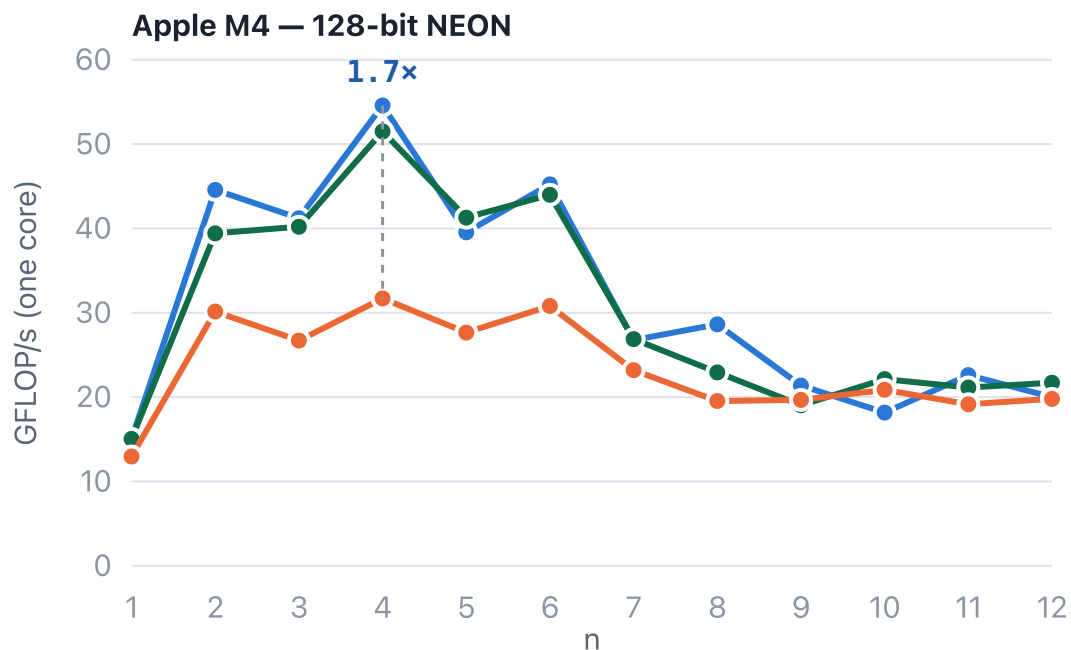


# Same flops, different speed

One `HyperDual{n,n}` multiply does  $7n^2 + 6n + 1$  flops: exactly as many as the nested-`Dual` multiply on the same  $(1 + n)^2$  floats

## Benchmarking `*` for different $n$

■ `HyperDual{n,n}` ■ `HyperDual + SIMD.Vec` ■ nested `Dual{Dual}`



# Ask the CPU

LIKWID on Zen 4, one `a * b` at  $n = 8$ : the hardware retires 497 flops for every variant

| PER MULTIPLY      | HYPERDUAL{8,8} | + SIMD.VEC | NESTED DUAL 8x8 |
|-------------------|----------------|------------|-----------------|
| retired flops     | 497            | 497        | 497             |
| – as fused FMAs   | 80%            | 84%        | 0%              |
| instructions      | 98             | 73         | 366             |
| retired $\mu$ ops | 157            | 83         | 826             |
| L1D accesses      | 86             | 89         | 311             |
| integer macro-ops | 34             | 5          | 227             |
| cycles            | 68             | 63         | 207             |

AMD EPYC 9354, likwid 5.4.1: `RETIRED_SSE_AVX_FLOPS` by type, `RETIRED_UOPS`, `LS_DISPATCH`, `MACRO_OPS_DISPATCHED`

- A quarter of nested `Dual` 's  $\mu$ ops are address and lane bookkeeping, not math.

# Conclusions

- HyperHessians can give some speedups vs ForwardDiff (especially if threading makes sense)
- If using `DifferentiationInterface`, very easy to try it out
- Spending a little bit of time choosing the Chunk size can be worthwhile
- Counting flops is dead (like super dead, taken behind the woodshed dead...)
- Auto vectorizer can sometimes make a real mess, consider SIMD.jl
- Think about the layout for your data
- Experiment with performance counters (LIKWID, perf, VTune, etc)

# Questions?

`github.com/KristofferC/HyperHessians.jl`

`slides: kristofferc.github.io/HyperHessians.jl/slides`

